Unit 2 Assignment 2

Matrix Algebra & Systems of Equations — Group Report

Group members: Razan, Lamia, Sara, Ahmed

This report summarizes our manual calculations for the six missions and the NumPy results we used to verify them. Every manual answer matched the NumPy output.

Dataset

House	Area	Bedrooms	Age
H1	120	3	10
H2	150	4	8
H3	180	4	6
H4	210	5	4

Mission 1 — Understanding the Matrix

$$X = \begin{vmatrix} 120 & 3 & 10 \\ 150 & 4 & 8 \\ 180 & 4 & 6 \\ 210 & 5 & 4 \end{vmatrix}$$

- Shape: **(4, 3)** — **4 rows** and **3 columns**.
- Each **row** represents one house (H1–H4).
- Each **column** represents one feature (Area, Bedrooms, Age).

> **AI Insight:** In Machine Learning, each row is a **sample** (one example) and each column is a **feature** (one measured property).

NumPy: `X.shape → (4, 3)` ✓ **verified**

Mission 2 — Matrix Operations

Operation	Manual result	NumPy
Age + 2	[12, 10, 8, 6]	✓
Age − 1	[9, 7, 5, 3]	✓
Area × 1.1	[132, 165, 198, 231]	✓
Transpose	shape (4, 3) → (3, 4)	✓

$$X^T = \begin{vmatrix} 120 & 150 & 180 & 210 \\ 3 & 4 & 4 & 5 \\ 10 & 8 & 6 & 4 \end{vmatrix}$$

AI Insight:

- **Operations that change values:** addition, subtraction and multiplication. The numbers change but the shape stays (4, 3).
- **Operation that changes structure:** transpose. The shape changes from (4, 3) to (3, 4).
- **Why transpose is useful:** it switches rows and columns so shapes match for matrix multiplication, for example X^TX in linear regression.

Mission 3 — AI Prediction

Weights: $w = [2, 20, -5]^T$ → **Score = X · w = 2·Area + 20·Bedrooms − 5·Age**
$(4\times3)\cdot(3\times1) = (4\times1)$

House	Manual calculation	Score
H1	2(120) + 20(3) − 5(10) = 240 + 60 − 50	**250**
H2	2(150) + 20(4) − 5(8) = 300 + 80 − 40	**340**
H3	2(180) + 20(4) − 5(6) = 360 + 80 − 30	**410**
H4	2(210) + 20(5) − 5(4) = 420 + 100 − 20	**500**

NumPy: `X @ w → [250, 340, 410, 500]` ✓ **verified**

AI Insight: Matrix multiplication is used in linear regression (prediction = X·w), in every layer of a neural network (output = W·x + b), and in recommendation systems and image processing. It computes the predictions for all samples at once.

Mission 4 — Detecting Redundant Features

New feature: **Rooms = Bedrooms × 2** → Rooms = [6, 8, 8, 10]. The new matrix has shape (4, 4).

NumPy: `np.linalg.matrix_rank(X_new) → 3` ✓ **verified**

Findings: The matrix has **4 columns but rank 3**. Because the rank is less than the number of columns, one column is a linear combination of another: Rooms is just Bedrooms × 2. It adds **no new information**, so it is a redundant feature.

> **AI Insight:** Redundant features are harmful because they make the model bigger and slower without adding information, can lead to overfitting, and make the matrix singular (det = 0), so methods that need an inverse, like linear regression, become unstable.

Mission 5 — Determinant, Identity Matrix & Inverse

$$A = \begin{vmatrix} 2 & 1 \\ 4 & 2 \end{vmatrix} \quad B = \begin{vmatrix} 2 & 1 \\ 3 & 2 \end{vmatrix} \quad I = \begin{vmatrix} 1 & 0 \\ 0 & 1 \end{vmatrix}$$

Formula: det [a b; c d] = **ad − bc**

- det(A) = (2)(2) − (1)(4) = 4 − 4 = **0** ✓ **NumPy**
- det(B) = (2)(2) − (1)(3) = 4 − 3 = **1** ✓ **NumPy**
- A × I = A ✓ and B × I = B ✓ — multiplying by the identity matrix changes nothing.

Invertible matrix: B, because det(B) = 1 ≠ 0.

$$B^{-1} = (1 / \det B) \cdot \begin{vmatrix} d & -b \\ -c & a \end{vmatrix} = \begin{vmatrix} 2 & -1 \\ -3 & 2 \end{vmatrix}$$ ✓ **NumPy** (and $B \cdot B^{-1} = I$)

Why A has no inverse: det(A) = 0. Row 2 of A is 2 × row 1 ([4, 2] = 2 × [2, 1]), so the rows are dependent. The inverse formula divides by the determinant, and we cannot divide by 0.

> **AI Insight:**
> - **det = 0** means the matrix is **singular**: its rows or columns are dependent, and it squashes space into a lower dimension, so information is lost.
> - **A singular matrix can't be inverted** because the lost information can't be recovered, and the formula would divide by zero.
> - **The inverse is important in ML** because it is used to find the best weights, for example in linear regression: $w = (X^T X)^{-1} X^T y$.

Mission 6 — Solving Linear Systems

System: $2x + y = 7$ and $x - y = 2$

Matrix form (A·x = b):

$$\begin{bmatrix} 2 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 7 \\ 2 \end{bmatrix}$$

Manual solution (elimination)

- Add the two equations: $(2x + y) + (x - y) = 7 + 2 \rightarrow 3x = 9 \rightarrow$ **x = 3**
- Substitute into $x - y = 2$: $3 - y = 2 \rightarrow$ **y = 1**

NumPy solution

`np.linalg.solve(A, b) → x = 3.0, y = 1.0`

Comparison: manual (x = 3, y = 1) and NumPy (x = 3, y = 1) give **the same solution** ✓

> **AI Insight:** Training a model means finding weights that satisfy many equations at once (one per data sample). Linear regression is solved as a system of equations ($X^TX \cdot w = X^Ty$), so solving systems is at the core of Machine Learning.

Conclusion

We represented real data as a matrix, applied element-wise operations and the transpose, used matrix multiplication to make predictions, detected a redundant feature using the rank, used the determinant to decide which matrix has an inverse, and solved a linear system both manually and with NumPy. All manual calculations were verified with NumPy and matched.